ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

ΣΧΕΔΙΑΣΗ ΚΑΙ ΥΛΟΠΟΙΗΣΗ ΣΥΣΤΗΜΑΤΟΣ ΕΠΕΞΕΡΓΑΣΙΑΣ ΔΕΔΟΜΕΝΩΝ IOT ΜΕ ΧΡΗΣΗ ΑΡΧΙΤΕΚΤΟΝΙΚΗΣ LAMBDA ΚΑΙ ΜΙΚΡΟΥΠΗΡΕΣΙΩΝ ΣΕ ΠΕΡΙΒΑΛΛΟΝ CLOUD

Προγραμματισμός και Υποδομές Μεγάλων Δεδομένων: Python και Νεφοϋπολογιστική

Ονοματεπώνυμο: Θεόδωρος Τσούτσουρας

Αριθμός Μητρώου: me25040

Επιβλέπων Καθηγητής: Γ. Κουσιουρής

Φεβρουάριος 2026

Περιεχόμενα

1. Εισαγωγή 2
2. Τεχνολογικό Πλαίσιο & Θεωρητική Προσέγγιση 3
2.1 Η Πρόκληση των Δεδομένων IoT 3
2.2.1 Batch Layer (Επίπεδο Δέσμης) 3
2.2.2 Speed Layer (Επίπεδο Ταχύτητας) 3
2.2.3 Serving Layer (Επίπεδο Εξυπηρέτησης) 4
2.3 Αρχιτεκτονική Μικρό-υπηρεσιών (Microservices) και Τεχνολογία Docker ... 5
3. Ανάλυση Αρχιτεκτονικής Συστήματος 6
3.1 Ροή Δεδομένων 6
3.2 Επιλογή και Ρόλος Εργαλείων 6
4. Υλοποίηση Batch Layer (Επίπεδο Δέσμης) 6
4.1 Υπηρεσία Ingestion 6
4.2 Αποθήκευση Δεδομένων (Object Storage) 6
5. Υλοποίηση Serving Layer (Επίπεδο Εξυπηρέτησης) 7
6. Υλοποίηση Speed Layer (Επίπεδο Ταχύτητας) 8
6.1 Ενορχήστρωση Ροής Δεδομένων (Orchestration) 8
6.2 Επιχειρησιακή Λογική Σύγκρισης (Business Logic) 9
6.3 Έξοδος Συστήματος 10
7. Διασύνδεση και Μηνύματα (Messaging Middleware) 12
7.1 Ο Ρόλος του RabbitMQ στην Αρχιτεκτονική 12
7.2 Τεχνική Υλοποίηση Δρομολόγησης 12
7.3 Από-σύζευξη και Ανθεκτικότητα (Decoupling) 13
8. Ενορχήστρωση Υποδομής (Docker Compose) 13
8.1 Εικονική Δικτύωση και Ανακάλυψη Υπηρεσιών (Service Discovery) 13
8.2 Διαχείριση Δεδομένων (Data Persistence) 14
8.3 Χαρτογράφηση Θυρών (Port Mapping) 14
9. Συμπεράσματα 16
9.1 Αξιολόγηση Αρχιτεκτονικής 16
9.2 Ο Ρόλος της Τεχνολογίας Docker 16
9.3 Σημασία της Από-σύζευξης (Decoupling) 16
10. Βιβλιογραφία 18

1. Εισαγωγή

Η ραγδαία εξάπλωση του Διαδικτύου των Πραγμάτων (IoT) έχει δημιουργήσει την ανάγκη για συστήματα ικανά να διαχειριστούν τεράστιους όγκους δεδομένων που παράγονται με υψηλή ταχύτητα. Οι παραδοσιακές αρχιτεκτονικές βάσεων δεδομένων συχνά αδυνατούν να ανταποκριθούν ταυτόχρονα σε δύο αντικρουόμενες απαιτήσεις: την ανάγκη για βαθιά ανάλυση ιστορικών στοιχείων και την ανάγκη για άμεση αντίδραση σε κρίσιμα γεγονότα.

Στο πλαίσιο αυτό, η παρούσα εργασία προτείνει την υλοποίηση μιας Lambda Architecture. Το σύστημα που αναπτύχθηκε παρακολουθεί τη θερμοκρασία της περιοχής των Αθηνών, αντλώντας δεδομένα από το OpenMeteo API. Στόχος είναι η δημιουργία ενός αγωγού δεδομένων (data pipeline) που αποθηκεύει τα δεδομένα για μελλοντική χρήση, ενώ ταυτόχρονα ελέγχει σε πραγματικό χρόνο αν η τρέχουσα θερμοκρασία αποκλίνει σημαντικά από τον ιστορικό μέσο όρο, ενεργοποιώντας τους κατάλληλους μηχανισμούς ειδοποίησης.

2. Τεχνολογικό Πλαίσιο & Θεωρητική Προσέγγιση

2.1 Η Πρόκληση των Δεδομένων IoT

Τα δεδομένα IoT χαρακτηρίζονται από τις "3 Vs" του Big Data: Volume (Όγκος), Velocity (Ταχύτητα) και Variety (Ποικιλία). Η αποθήκευση αυτών των δεδομένων σε μια σχεσιακή βάση (RDBMS) και η ταυτόχρονη εκτέλεση πολύπλοκων ερωτημάτων (queries) μπορεί να οδηγήσει σε συμφόρηση (bottleneck) και αυξημένη καθυστέρηση (latency), καθιστώντας το σύστημα ακατάλληλο για εφαρμογές πραγματικού χρόνου.

2.2.1 Batch Layer (Επίπεδο Δέσμης)

Το Batch Layer αποτελεί το θεμέλιο της αρχιτεκτονικής του συστήματος.

- Λειτουργία: Αποθηκεύει ένα αμετάβλητο (immutable) σύνολο δεδομένων, το οποίο διαρκώς αυξάνεται (append-only). Αυτό σημαίνει ότι τα δεδομένα που εισέρχονται δεν διαγράφονται και δεν τροποποιούνται ποτέ.
- Σκοπός: Επειδή διαχειρίζεται το σύνολο των ιστορικών δεδομένων, μπορεί να παράγει αποτελέσματα (Views) με απόλυτη ακρίβεια, διορθώνοντας τυχόν λάθη που έγιναν στο παρελθόν μέσω νέου υπολογισμού.
- Χαρακτηριστικά: Υψηλή ακρίβεια, δυνατότητα διαχείρισης μεγάλου όγκου (Big Data), αλλά και υψηλή καθυστέρηση (High Latency), καθώς η επεξεργασία όλου του ιστορικού απαιτεί χρόνο.
- Στην εργασία μας: Υλοποιείται μέσω του Minio (αποθήκευση JSON αρχείων) και του Python Ingestion Script.

2.2.2 Speed Layer (Επίπεδο Ταχύτητας)

Το Speed Layer υπάρχει για να καλύψει το «χρονικό κενό» που αφήνει το Batch Layer.

- Λειτουργία: Επεξεργάζεται τα δεδομένα τη στιγμή που φτάνουν (stream processing) και δεν ασχολείται με το βαθύ ιστορικό. Λειτουργεί σαν «παράθυρο» που βλέπει μόνο τα πολύ πρόσφατα γεγονότα.
- Σκοπός: Να παρέχει απαντήσεις σε πραγματικό χρόνο. Αν περιμέναμε το Batch Layer να τρέξει (π.χ. μία φορά την ώρα), θα χάναμε κρίσιμα γεγονότα που συμβαίνουν τώρα. Το Speed Layer θυσιάζει την απόλυτη ακρίβεια και την πολυπλοκότητα για χάρη της ταχύτητας.
- Χαρακτηριστικά: Ελάχιστη καθυστέρηση (Low Latency), άμεση απόκριση, προσωρινή φύση αποτελεσμάτων.

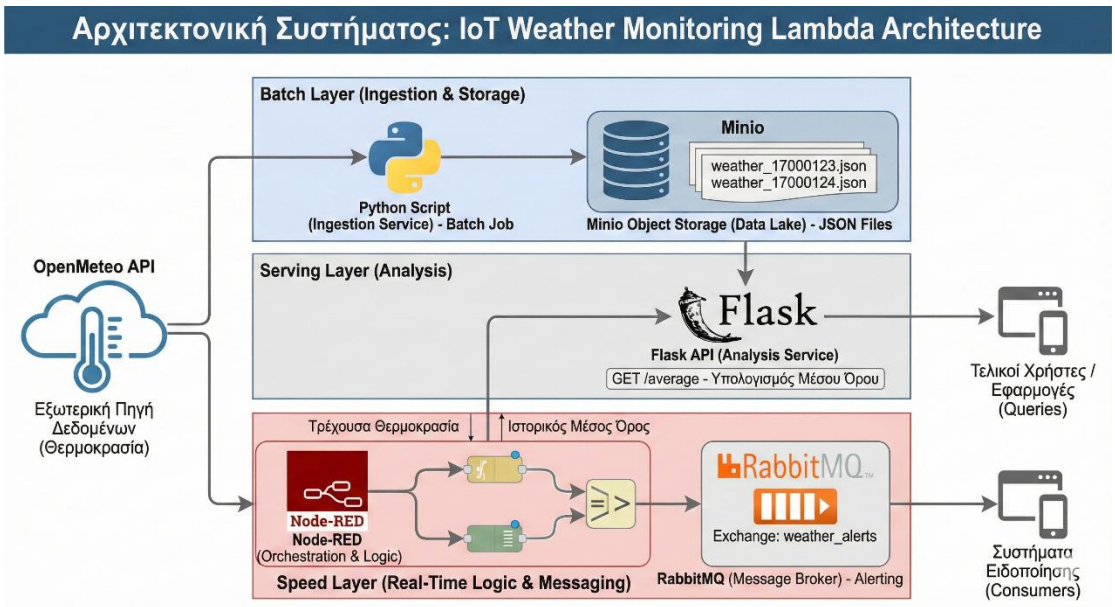
- Στην εργασία μας: Υλοποιείται μέσω του Node-RED, το οποίο λαμβάνει την τρέχουσα θερμοκρασία και αποφασίζει άμεσα (Alerting) χωρίς να περιμένει την αποθήκευση.

2.2.3 Serving Layer (Επίπεδο Εξυπηρέτησης)

Το Serving Layer είναι το σημείο επαφής του συστήματος με τον έξω κόσμο.

- Λειτουργία: Δεν κάνει πολύπλοκους υπολογισμούς ούτε δέχεται νέα δεδομένα απευθείας. Αντίθετα, φορτώνει τα προ-υπολογισμένα αποτελέσματα (Views) από το Batch Layer και τα κάνει αναζητήσασα (indexed) για γρήγορη πρόσβαση.
- Σκοπός: Να απαντά σε ερωτήματα χρηστών (queries) άμεσα. Συνδυάζει την αξιοπιστία του Batch Layer με την ταχύτητα ανάγνωσης μιας βάσης δεδομένων.
- Χαρακτηριστικά: Χαμηλή καθυστέρηση στην ανάγνωση (Low Latency Reads), επεκτασιμότητα.
- Στην εργασία μας: Υλοποιείται μέσω του Flask API, το οποίο διαβάζει τα αρχεία του Minio και επιστρέφει τον υπολογισμένο μέσο όρο όταν του ζητηθεί.

Εικόνα [1]: Αρχιτεκτονικό διάγραμμα ροής δεδομένων και διασύνδεσης εργαλείων του συστήματος.



2.3 Αρχιτεκτονική Μικρό-υπηρεσιών (Microservices) και Τεχνολογία Docker

Σε αντίθεση με τις παραδοσιακές "μονολιθικές" (monolithic) εφαρμογές, όπου όλες οι λειτουργίες ενσωματώνονται σε έναν ενιαίο εκτελέσιμο κώδικα, η παρούσα εργασία υιοθετεί την Αρχιτεκτονική Μικρό-υπηρεσιών (Microservices Architecture). Σύμφωνα με αυτή την προσέγγιση, το σύστημα διασπάται σε μικρότερες, αυτόνομες μονάδες λογισμικού, οι οποίες επικοινωνούν μεταξύ τους μέσω καλά ορισμένων διεπαφών (API).

Για την υλοποίηση αυτής της αρχιτεκτονικής, επιλέχθηκε η πλατφόρμα Docker, η οποία επιτρέπει την εκτέλεση εφαρμογών μέσα σε απομονωμένα περιβάλλοντα που ονομάζονται "Εμπορευματοκιβώτια" (Containers).

Η χρήση του Docker προσφέρει στην εργασία τα εξής κρίσιμα πλεονεκτήματα:

α) Διαχείριση Εξαρτήσεων (Dependency Management): Κάθε υπηρεσία της Lambda Architecture έχει διαφορετικές απαιτήσεις. Για παράδειγμα, το *Ingestion Script* απαιτεί βιβλιοθήκες Python, το *Node-RED* απαιτεί περιβάλλον Node.js, ενώ το *Minio* είναι ένα εκτελέσιμο γραμμένο σε Go. Με τη χρήση containers, κάθε υπηρεσία "πακετάρετε" μαζί με τις δικές της βιβλιοθήκες και εκδόσεις λογισμικού, εξαλείφοντας το πρόβλημα ασυμβατότητας ("Dependency Hell") που συχνά εμφανίζεται όταν πολλές εφαρμογές τρέχουν στο ίδιο λειτουργικό σύστημα.

β) Απομόνωση και Σταθερότητα (Isolation): Τα containers παρέχουν απομόνωση σε επίπεδο διεργασιών και συστήματος αρχείων. Εάν, για παράδειγμα, η υπηρεσία ανάλυσης (Flask) παρουσιάσει σφάλμα και τερματιστεί, αυτό δεν θα επηρεάσει τη λειτουργία του RabbitMQ ή του Minio. Το σύστημα παραμένει μερικώς λειτουργικό, αυξάνοντας τη συνολική διαθεσιμότητα (Availability).

γ) Τυποποίηση Περιβάλλοντος (Environment Consistency): Το Docker διασφαλίζει ότι ο κώδικας θα εκτελείται πανομοιότυπα σε οποιοδήποτε μηχάνημα (Local Laptop, Server, Cloud VM). Μέσω των αρχείων Dockerfile και docker-compose.yml, ορίζεται επακριβώς η υποδομή ως κώδικας (Infrastructure as Code - IaC), επιτρέποντας την άμεση αναπαραγωγή του πειράματος από οποιονδήποτε χρήστη χωρίς την ανάγκη πολύπλοκων ρυθμίσεων.

δ) Εσωτερική Δικτύωση (Service Discovery): Το Docker δημιουργεί ένα εικονικό δίκτυο (στην περίπτωσή μας το cloud_net), το οποίο επιτρέπει στις υπηρεσίες να αναγνωρίζουν η μία την άλλη μέσω DNS ονομάτων (π.χ. minio, rabbitmq_server), καταργώντας την ανάγκη για διαχείριση στατικών IP διευθύνσεων.

3. Ανάλυση Αρχιτεκτονικής Συστήματος

3.1 Ροή Δεδομένων

Η ροή της πληροφορίας στο σύστημα ακολουθεί δύο παράλληλους δρόμους:

1. Cold Path (Batch): API -> Python Script -> Minio -> Flask API. Στόχος η ιστορικότητα.
2. Hot Path (Speed): API -> Node-RED -> RabbitMQ. Στόχος η αμεσότητα.

Τα δύο μονοπάτια συγκλίνουν στο Node-RED, όπου η "τρέχουσα γνώση" (Speed Layer) συγκρίνεται με την "ιστορική γνώση" (Serving Layer).

3.2 Επιλογή και Ρόλος Εργαλείων

Η επιλογή των εργαλείων δεν έγινε τυχαία, αλλά με βάση τον λειτουργικό τους ρόλο:

- Minio (Object Storage): Λειτουργεί ως το Data Lake της αρχιτεκτονικής. Επιλέχθηκε για τη συμβατότητά του με το πρότυπο S3 και την ικανότητά του να αποθηκεύει αδόμητα δεδομένα (JSON files) με ασφάλεια.
- RabbitMQ (Message Broker): Αποτελεί το ενδιάμεσο λογισμικό (middleware) για την ασύγχρονη επικοινωνία. Ο ρόλος του είναι κρίσιμος για την από-σύζευξη (decoupling) του συστήματος απόφασης (Node-RED) από τα συστήματα κατανάλωσης ειδοποιήσεων.
- Node-RED: Αποτελεί τον ενορχηστρωτή του Speed Layer. Η visual programming φύση του επιτρέπει την ταχεία ανάπτυξη πολύπλοκων ροών λογικής και τη εύκολη διασύνδεση διαφορετικών API.

4. Υλοποίηση Batch Layer (Επίπεδο Δέσμης)

4.1 Υπηρεσία Ingestion

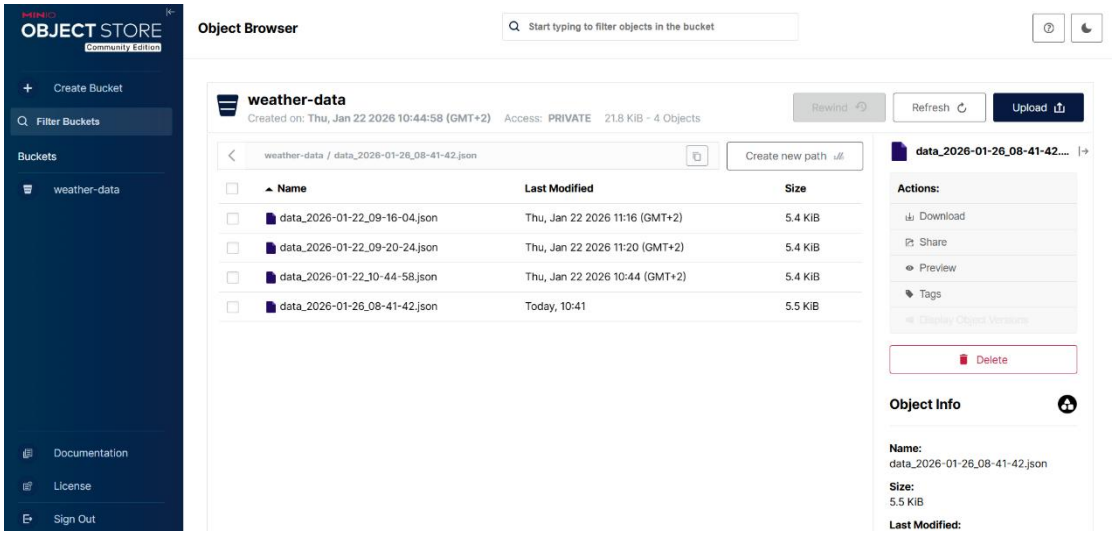
Για την εισαγωγή των δεδομένων, αναπτύχθηκε ένα script σε γλώσσα Python. Το script αυτό λειτουργεί ως Batch Job: εκτελείται, συνδέεται στο OpenMeteo API, ανακτά τα μετεωρολογικά δεδομένα σε μορφή JSON και τερματίζει. Η διαδικασία αυτή επαναλαμβάνεται περιοδικά.

4.2 Αποθήκευση Δεδομένων (Object Storage)

Τα δεδομένα αποθηκεύονται στο Minio. Κάθε λήψη δεδομένων αποθηκεύεται ως ξεχωριστό αρχείο JSON με μοναδικό όνομα βασισμένο στο timestamp (π.χ.

`weather_17000123.json`). Αυτή η στρατηγική διασφαλίζει ότι διατηρούμε τα πρωτογενή δεδομένα (Raw Data) αναλλοίωτα, επιτρέποντας την επανάληψη της επεξεργασίας στο μέλλον με διαφορετικούς αλγορίθμους αν χρειαστεί.

Εικόνα [2]: Το γραφικό περιβάλλον διαχείρισης του Minio (Object Browser). Διακρίνεται ο κάδος αποθήκευσης weather-data και η λίστα των JSON αρχείων που έχουν εισαχθεί επιτυχώς από την υπηρεσία Ingestion, επιβεβαιώνοντας τη λειτουργία του Batch Layer.



5. Υλοποίηση Serving Layer (Επίπεδο Εξυπηρέτησης)

Το Serving Layer υλοποιήθηκε με χρήση του micro-framework Flask (Python). Ο ρόλος του είναι να μετατρέπει τα "ακατέργαστα" δεδομένα που βρίσκονται στο Minio σε χρήσιμη και προσβάσιμη πληροφορία για άλλες εφαρμογές.

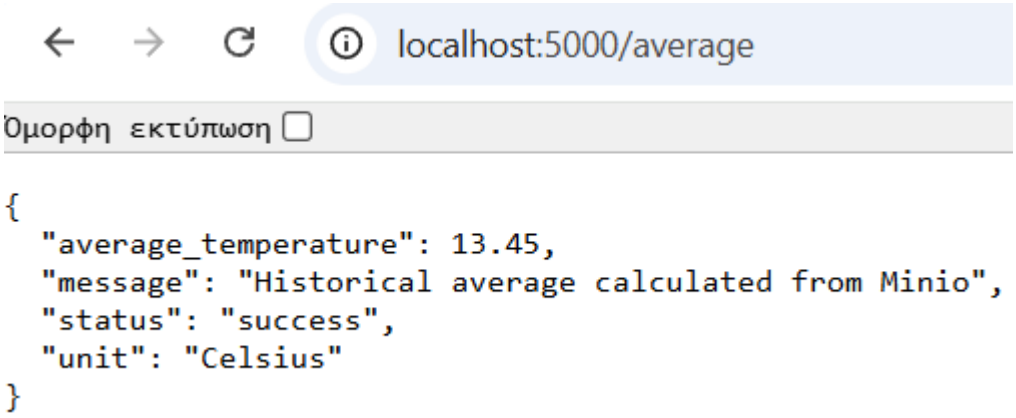
Ανάλυση Λειτουργίας API

Η υπηρεσία εκθέτει ένα RESTful endpoint στη διαδρομή: GET /average. Κατά την κλήση αυτού του endpoint, εκτελούνται τα εξής βήματα:

1. Ανάκτηση: Η υπηρεσία συνδέεται στο Minio μέσω του S3 API και ανακτά τη λίστα όλων των αποθηκευμένων JSON αρχείων.
2. Ανάγνωση: Διατρέχει (iterate) κάθε αρχείο, κάνει parse το περιεχόμενο JSON και εξάγει την τιμή της θερμοκρασίας (temperature).
3. Υπολογισμός: Υπολογίζει τον αριθμητικό μέσο όρο όλων των ιστορικών εγγραφών.
4. Απόκριση: Επιστρέφει το αποτέλεσμα σε μορφή JSON: {"average_temperature": XX.X}.

Αυτή η προσέγγιση επιτρέπει στο Speed Layer (ή σε οποιονδήποτε άλλο πελάτη) να έχει πρόσβαση στην "ιστορική γνώση" του συστήματος μέσω μιας απλής HTTP κλήσης, χωρίς να απαιτείται γνώση της δομής αποθήκευσης.

Εικόνα [3]: Στιγμιότυπο απόκρισης του Serving Layer (Flask API). Το σύστημα επιστρέφει αντικείμενο JSON που περιέχει τον υπολογισμένο ιστορικό μέσο όρο θερμοκρασίας (13.45°C), επιβεβαιώνοντας την επιτυχή ανάκτηση δεδομένων από το Minio.



6. Υλοποίηση Speed Layer (Επίπεδο Ταχύτητας)

Το Speed Layer αποτελεί το τμήμα της αρχιτεκτονικής που είναι υπεύθυνο για την επεξεργασία δεδομένων σε πραγματικό χρόνο (Real-time Processing). Σε αντίθεση με το Batch Layer, το οποίο εστιάζει στην ακρίβεια και την ιστορικότητα, το Speed Layer εστιάζει στην χαμηλή καθυστέρηση (Low Latency).

Για την υλοποίηση αυτού του επιπέδου επιλέχθηκε το εργαλείο Node-RED. Πρόκειται για ένα εργαλείο ροής προγραμματισμού (Flow-based programming tool), το οποίο επιτρέπει την οπτική διασύνδεση υπηρεσιών (APIs) και συσκευών IoT.

6.1 Ενορχήστρωση Ροής Δεδομένων (Orchestration)

Η ροή εργασίας (workflow) που σχεδιάστηκε στο Node-RED εκτελείται περιοδικά ανά 10 δευτερόλεπτα (Trigger Node) και ακολουθεί τα εξής βήματα:

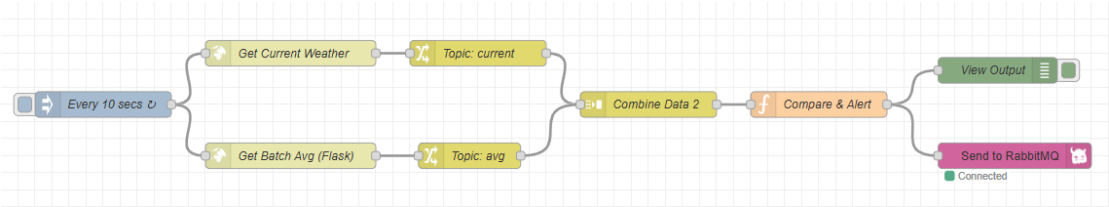
1. Παράλληλη Ανάκτηση Δεδομένων: Με την έναρξη του κύκλου, το σύστημα εκτελεί δύο ταυτόχρονα HTTP GET αιτήματα:

    - Πηγή A (Real-time): Καλεί το OpenMeteo API για να λάβει την τρέχουσα θερμοκρασία της Αθήνας.

• Πηγή B (Historical Context): Καλεί το εσωτερικό Flask API (Serving Layer) για να λάβει τον ιστορικό μέσο όρο θερμοκρασίας, όπως αυτός έχει υπολογιστεί από το Batch Layer.

2. Συγχρονισμός (Synchronization): Δεδομένου ότι οι δύο παραπάνω κλήσεις είναι ασύγχρονες και ενδέχεται να ολοκληρωθούν σε διαφορετικούς χρόνους, χρησιμοποιήθηκε ο κόμβος Join. Ο κόμβος αυτός περιμένει να λάβει απαντήσεις και από τις δύο πηγές πριν προωθήσει τα δεδομένα στο επόμενο στάδιο, διασφαλίζοντας ότι η σύγκριση θα γίνει με πλήρη δεδομένα.

Εικόνα [4]: Το διάγραμμα ροής (Flow) στο Node-RED. Απεικονίζεται η ταυτόχρονη λήψη δεδομένων, η συνένωσή τους και η λογική σύγκρισης.



6.2 Επιχειρησιακή Λογική Σύγκρισης (Business Logic)

Στον πυρήνα του Speed Layer βρίσκεται ένας κόμβος επεξεργασίας (Function Node) που εκτελεί κώδικα JavaScript. Ο ρόλος του είναι να υλοποιήσει τον αλγόριθμο ανίχνευσης ανωμαλιών (Anomaly Detection).

Συγκεκριμένα, ο κώδικας συγκρίνει την Τρέχουσα Θερμοκρασία (T_{curr}) με τον Ιστορικό Μέσο Όρο (T_{avg}). Εάν η απόλυτη διαφορά τους υπερβαίνει ένα καθορισμένο κατώφλι (threshold), το σύστημα κρίνει το γεγονός ως άξιο αναφοράς.

Ο κώδικας δημιουργεί δυναμικά ένα αντικείμενο JSON (Alert Object) με την εξής δομή:

Εικόνα[5]:JSON(AlertObject).

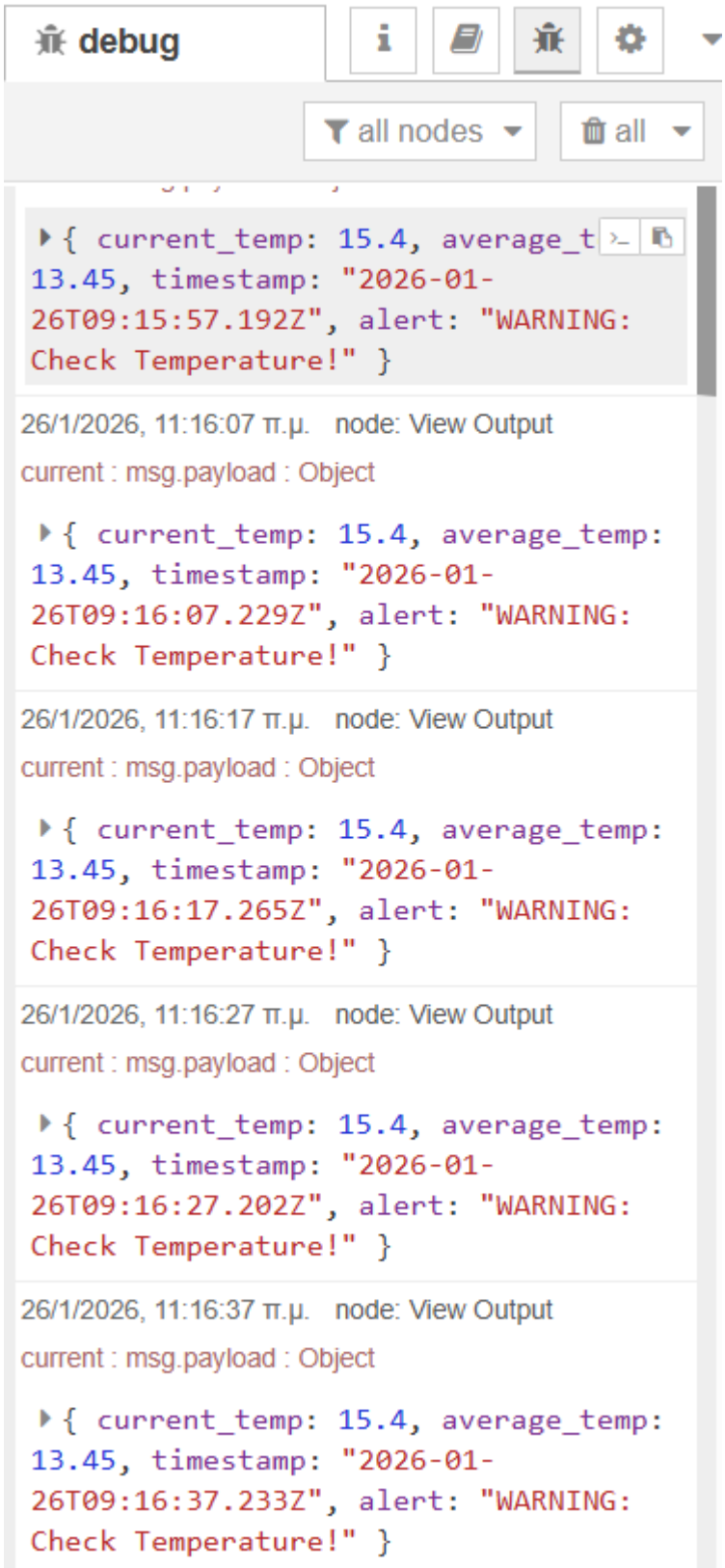
```
{
    "alert": "Temperature Deviation",
    "current_temp": 15.0,
    "average_temp": 13.45,
    "timestamp": "2024-01-26T10:30:00Z"
}
```

6.3 Έξοδος Συστήματος

Το παραγόμενο αντικείμενο ειδοποίησης διοχετεύεται σε δύο εξόδους:

1. Debug Console: Για άμεση επιβεβαίωση της λειτουργίας από τον διαχειριστή.
2. RabbitMQ Output: Για την ασύγχρονη αποστολή του μηνύματος προς κατανάλωση από άλλες υπηρεσίες.

Εικόνα [6]: Το παράθυρο αποσφαλμάτωσης (Debug) του Node-RED, όπου επιβεβαιώνεται η δημιουργία του αντικειμένου ειδοποίησης σε πραγματικό χρόνο.



7. Διασύνδεση και Μηνύματα (Messaging Middleware)

Για την επίτευξη της επικοινωνίας μεταξύ του Speed Layer (Node-RED) και των τελικών καταναλωτών της πληροφορίας (π.χ. συστήματα ειδοποίησης), επιλέχθηκε η χρήση ενός Message Broker. Συγκεκριμένα, υιοθετήθηκε το λογισμικό RabbitMQ, το οποίο υλοποιεί το πρωτόκολλο AMQP (Advanced Message Queuing Protocol).

7.1 Ο Ρόλος του RabbitMQ στην Αρχιτεκτονική

Σε μια κατανεμημένη αρχιτεκτονική, η άμεση επικοινωνία (point-to-point) μεταξύ των υπηρεσιών δημιουργεί ισχυρές εξαρτήσεις. Αν, για παράδειγμα, το σύστημα αποστολής email πέσει, το Node-RED θα κολλούσε περιμένοντας απόκριση.

Το RabbitMQ λύνει αυτό το πρόβλημα λειτουργώντας ως ενδιάμεσος (Middleware):

1. Παραγωγός (Producer): Το Node-RED στέλνει το μήνυμα JSON (Alert Object) στο RabbitMQ και "ξεχνάει" την ύπαρξή του, συνεχίζοντας απρόσκοπτα τη λειτουργία του.
2. Δρομολόγηση (Routing): Το μήνυμα παραλαμβάνεται από ένα Exchange με το όνομα weather_alerts.
3. Ουρά (Queue): Το RabbitMQ αποθηκεύει το μήνυμα σε μια ουρά μέχρι να βρεθεί διαθέσιμος παραλήπτης να το επεξεργαστεί.

7.2 Τεχνική Υλοποίηση Δρομολόγησης

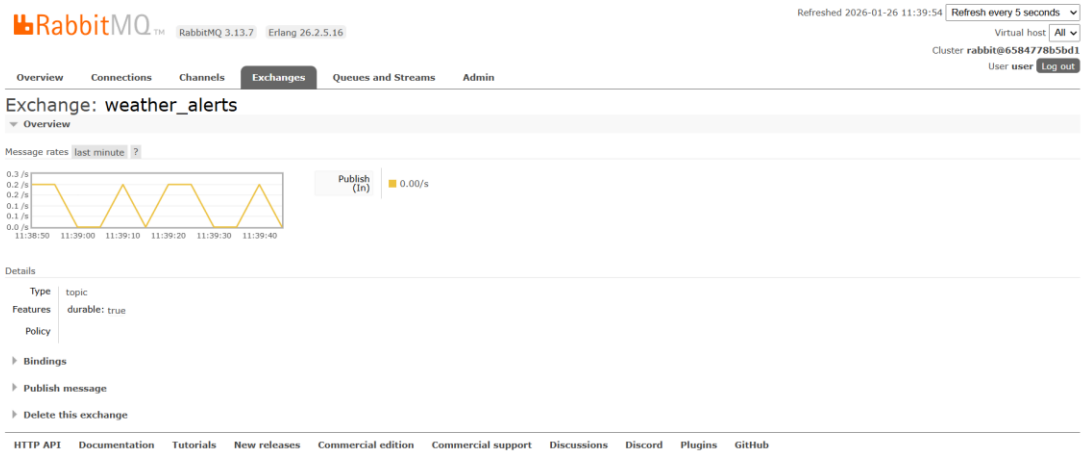
Για την εργασία χρησιμοποιήθηκε η μέθοδος Topic Exchange.

• Exchange Name: weather_alerts

• Routing Key: athens.temp

Η χρήση του Routing Key επιτρέπει μελλοντική επέκταση. Για παράδειγμα, θα μπορούσαμε να έχουμε κλειδιά όπως london.temp ή athens.humidity, και διαφορετικές υπηρεσίες να "ακούν" μόνο τα μηνύματα που τις ενδιαφέρουν, χωρίς να αλλάξουμε τον κώδικα του αποστολέα.

Εικόνα [7]: Το περιβάλλον διαχείρισης του RabbitMQ. Διακρίνεται το Exchange weather_alerts καθώς και η γραφική παράσταση της ροής μηνυμάτων, η οποία επιβεβαιώνει την ενεργή λήψη δεδομένων από το Node-RED.



7.3 Από-σύζευξη και Ανθεκτικότητα (Decoupling)

Η σημαντικότερη συνεισφορά του RabbitMQ στο σύστημα είναι η Ασύγχρονη Από-σύζευξη (Asynchronous Decoupling). Το Speed Layer (Node-RED) δεν γνωρίζει ποιος, πότε και αν θα διαβάσει το μήνυμα. Αυτό εξασφαλίζει ότι μια βλάβη στα συστήματα κατανάλωσης (Consumers) δεν θα επηρεάσει ποτέ τη ροή εισαγωγής και ανάλυσης δεδομένων, καθιστώντας την αρχιτεκτονική εξαιρετικά ανθεκτική (Fault Tolerant).

8. Ενορχήστρωση Υποδομής (Docker Compose)

Η διαχείριση πολλαπλών, αλληλεξαρτώμενων μικρό υπηρεσιών (microservices) αποτελεί μια σύνθετη διαδικασία. Για την αυτοματοποίηση της ανάπτυξης και της διασύνδεσης των υπηρεσιών του συστήματος, χρησιμοποιήθηκε το εργαλείο Docker Compose.

Μέσω ενός κεντρικού αρχείου ρυθμίσεων (docker-compose.yml), ορίζεται η "επιθυμητή κατάσταση" (desired state) ολόκληρης της υποδομής, επιτρέποντας την εκκίνηση του συστήματος με μία και μόνο εντολή (docker-compose up).

8.1 Εικονική Δικτύωση και Ανακάλυψη Υπηρεσιών (Service Discovery)

Ένα από τα σημαντικότερα στοιχεία της υλοποίησης είναι η δημιουργία ενός ιδιωτικού εικονικού δικτύου, το οποίο ονομάστηκε cloud_net. Όλα τα containers του συστήματος εντάχθηκαν σε αυτό το δίκτυο, προσφέροντας δύο κρίσιμα πλεονεκτήματα:

1. Απομόνωση (Isolation): Το δίκτυο είναι κλειστό. Οι υπηρεσίες επικοινωνούν μεταξύ τους, αλλά δεν είναι προσβάσιμες από άλλα containers που τυχόν τρέχουν στο ίδιο μηχάνημα, ενισχύοντας την ασφάλεια.
2. Αυτόματη Επίλυση Ονομάτων (DNS Resolution): Εντός του cloud_net, τα containers αναγνωρίζονται με το όνομά τους και όχι με IP διευθύνσεις. Για παράδειγμα, το Node-RED συνδέεται στο http://rabbitmq_server:5672. Το Docker αναλαμβάνει αυτόματα να μεταφράσει το όνομα rabbitmq_server στη σωστή εσωτερική IP, ανεξάρτητα από το πώς ή πότε ξεκίνησε το container.

8.2 Διαχείριση Δεδομένων (Data Persistence)

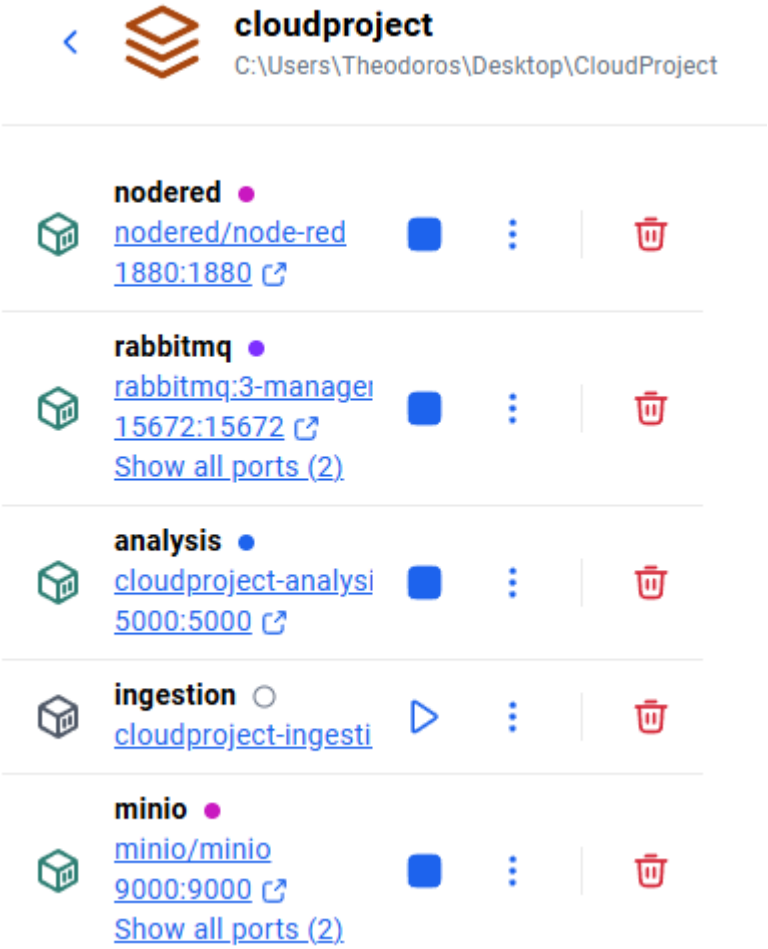
Δεδομένου ότι τα containers είναι από τη φύση τους "εφήμερα" (ephemeral) — δηλαδή τα δεδομένα τους χάνονται όταν τερματιστούν— χρησιμοποιήθηκε ο μηχανισμός των Docker Volumes. Συγκεκριμένα, για το Minio ορίστηκε ο τόμος minio_data. Αυτό διασφαλίζει ότι τα αποθηκευμένα αρχεία JSON (το Batch Layer) διατηρούνται μόνιμα στον δίσκο του συστήματος υποδοχής (Host), ακόμα και αν διαγραφεί ή επανεκκινηθεί το container του Minio.

8.3 Χαρτογράφηση Θυρών (Port Mapping)

Για να είναι εφικτή η διαχείριση και η εποπτεία του συστήματος από τον χρήστη, πραγματοποιήθηκε χαρτογράφηση των εσωτερικών θυρών των containers σε θύρες του τοπικού υπολογιστή (localhost):

- Minio Console: Θύρα 9001 (Διαχείριση Αρχείων)
- RabbitMQ Management: Θύρα 15672 (Παρακολούθηση Μηνυμάτων)
- Node-RED UI: Θύρα 1880 (Σχεδιασμός Ροών)
- Flask API: Θύρα 5000 (Εξυπηρέτηση Αιτημάτων)

Εικόνα [8]: Στιγμιότυπο από το Docker Desktop που απεικονίζει την κατάσταση των μικρό υπηρεσιών. Διακρίνονται οι υπηρεσίες minio, rabbitmq, nodered και analysis σε κατάσταση λειτουργίας (Running), καθώς και η υπηρεσία ingestion η οποία έχει ολοκληρώσει επιτυχώς τον κύκλο εκτέλεσής της.



9. Συμπεράσματα

Η παρούσα εργασία ολοκλήρωσε με επιτυχία τον σχεδιασμό και την υλοποίηση ενός ολοκληρωμένου συστήματος επεξεργασίας δεδομένων, βασισμένου στις αρχές της Lambda Architecture και της τεχνολογίας των Microservices. Μέσα από την πρακτική εφαρμογή εργαλείων όπως το Docker, το Minio, το Flask, το Node-RED και το RabbitMQ, αναδείχθηκαν σημαντικά συμπεράσματα σχετικά με τη σύγχρονη Μηχανική Δεδομένων (Data Engineering).

9.1 Αξιολόγηση Αρχιτεκτονικής

Η επιλογή της διαίρεσης του συστήματος σε διακριτά επίπεδα (Batch Layer & Speed Layer) αποδείχθηκε καθοριστική για την αρτιότητα της λύσης:

Αξιοπιστία Δεδομένων: Το Batch Layer (Minio & Python Ingestion) διασφάλισε την ακεραιότητα και τη μόνιμη αποθήκευση των ιστορικών δεδομένων, δημιουργώντας μια "μοναδική πηγή αλήθειας" (Single Source of Truth).

Χαμηλή Καθυστέρηση: Το Speed Layer (Node-RED) επέτρεψε την άμεση λήψη αποφάσεων σε πραγματικό χρόνο, χωρίς να επιβαρύνεται από τον όγκο των ιστορικών δεδομένων, επιτυγχάνοντας ταχύτατους χρόνους απόκρισης.

9.2 Ο Ρόλος της Τεχνολογίας Docker

Η χρήση του Docker και του Docker Compose αποτέλεσε τον θεμέλιο λίθο της υλοποίησης. Προσέφερε:

• Φορητότητα (Portability): Το σύστημα μπορεί να μεταφερθεί και να εκτελεστεί σε οποιοδήποτε περιβάλλον (από ένα laptop μέχρι έναν cloud server) χωρίς αλλαγές στον κώδικα.

• Απομόνωση (Isolation): Κάθε υπηρεσία λειτούργησε με τις δικές της βιβλιοθήκες και εξαρτήσεις, εξαλείφοντας προβλήματα ασυμβατότητας ("Dependency Hell").

• Ευκολία Διαχείρισης: Η εκκίνηση όλης της υποδομής με μία εντολή απλοποίησε δραματικά τη διαδικασία ανάπτυξης και ελέγχου.

9.3 Σημασία της Από-σύζευξης (Decoupling)

Η ενσωμάτωση του RabbitMQ ως ενδιάμεσου φορέα επικοινωνίας απέδειξε την αξία της ασύγχρονης αρχιτεκτονικής. Το σύστημα είναι ανθεκτικό σε σφάλματα (Fault Tolerant), καθώς μια πιθανή προσωρινή αστοχία των καταναλωτών (Consumers) δεν επηρεάζει τη ροή συλλογής και επεξεργασίας δεδομένων, διασφαλίζοντας την επιχειρησιακή συνέχεια.

Συνοψίζοντας, η εργασία αυτή απέδειξε ότι η χρήση εργαλείων ανοιχτού κώδικα (Open Source) σε συνδυασμό με σύγχρονες Cloud-Native πρακτικές, επιτρέπει τη

δημιουργία ισχυρών, ευέλικτων και επεκτάσιμων συστημάτων IoT/Big Data με χαμηλό κόστος ανάπτυξης.

10. Βιβλιογραφία

1. Marz, N., & Warren, J. (2015). *Big Data: Principles and best practices of scalable real-time data systems*. Manning Publications.

2. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

3. Poulton, N. (2023). *Docker Deep Dive*.

4. Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.